

Synthesis (tts)

Profile of ability

Speech synthesis, also known as Text to Speech (TTS) technology, supports the real-time return of synthetic audio streaming data.

Request address

Singapore site access address: ws: // tts-api-sg.xf-yun .com /v 2/tts

The following address is for reference only, the specific access needs and product confirmation:

```
ws [s ]://tts -
api .xfyun .cn /v 2/tts ?authorization =aG 1hYyB 1c 2VybmFtZT 0iMTAwSU
1FIiwgYWxnb 3J pdGhtPSJobWFjLXNoYTI 1NiIsIGhlYWRLcnM 9Imhvc 3QgZGF 0ZSByZXF
1ZXN 0LWxpbnUiLCBz aWduYXR 1cmU 9IlVSbnk 4M 3o 1eIJsnWF 1ODl 1YXhUL
1dGdUtWejZVNkdkWDDV 25SMGdueWc 9I
g %3D %3D &date =Tue %2C +18+Dec +2018+09%3A 08%3A 49+UTC &host
=tts -api .xfyun . The n request parameters and implications:
```

- Host: the requested host, tts-api-sg.xf-yun.com
- Date: Current date, the UTC time zone
- authorization: Authenticative parameters, see the specific generation method_
Shake hands & authentication

Data upload and reception

- After a successful handshake, the client and the server will establish a websocket connection, and the client will be connected through the websock

It can both upload and receive data simultaneously. When the server has the result, the result will be pushed to the client through the websocket connection.

- The client only sends text data and parameters once a session, and the engine pushes synthetic results
client. When the engine data is synthesized, the end
identification is returned, as follows:

```
{
  ...# Other fields
  "data ":{
    ...# Other fields
    "status ":2
```

}
}

A connection support supports a one-way session that closes the connection

Example of the request parameters

```
{
  "common ":{
    "app _id " : "123456"
  },
  "business ":{
    "ent " : "intp 65",
    "vcn " : "xiaoyan ",
    "volume" : 50,
    "sp eed " : 50,
    "aue " : "speex -wb ;7",
    "auf " : "audio /L 16;rate =16000"
  },
  "data ":{
    "sta tus " : 2,
    "text " : "exSI 6ICJlbiIsCgkgICAgInBvc 2l 0aW 9uIjogImZhbnHnIigoJf ..."
  }
}
```

Request a parameter description

•common Parameter description:

In common, appid, uid and etc are mainly set, and more parameters are shown in common

•The * number parameter is for test or experience, only individual engines or a few speakers support, the function is unstable

And may be changed or offline at any time, the official release of the product do not use

•business Parameter description, mainly store some business parameters in business:

parameter name	type	Must pass	description	instance

e nt	s tri ng	yes	Engine type, not passed in xts 2.0, cancelled by speakers x3 and x4 and later. Optional value:	intp 65
------	-------------	-----	---	---------

parameter name	type	Must pass	description	instance
			aisoun d (Common effect) i ntp 65 (Chinese) intp 65_en (English) mts (small language, to be used with small language speakers) x (optimization effect) The default is to the int p 65 Personalized synthesis of pttts	
a ue	s tri ng	yes	Audio encoding, optional value: speex: compressed format (Default) speex-wb; 7: compression format (compression level 1~10, the default is 7, semicolon, after the compression level) op us: Compression format, opus 8k Opus-wb: Compressed format, opus 16k The opus-swb: compressed format, (is only supported by 24k speakers) l ame: mp 3 (using the parameter sfl = 1) speex-org-b (for speex _wideband, 16k) or aue = speex -org-nb (for speex_narrowband, 8k). Support similar	

			spe ex-org-bb; 7 transmission parameter specified compression level (default level 8) ico	
a uf	s tri ng	yes	Audio sampling rate, optional: auf = a udio / L 16; rate =8 000: Synthetic 8K audio auf = a udio / L 16; rate =1 6000: Synthetic 16K audio	audio/L16;rate = 16000

parameter name	type	Must pass	description	instance
			An auf = a udio / L 16; rate =2 4000: synthetic 24K audio No auf: 16K audio	
vcn	s tri ng	yes	Pronunciation, optional value: see speaker list for details	x iaoyan
s peed	i nt	den y	Speech Speed, optional value: [0- 100]	50
s pd	s tri ng	den y	(Like speed, the parameter value is different) speaking speed, optional value: [0 -10]	5
v olume	i nt	den y	Volume, optional value: [0- 100], with a default to 50	50
p itch	i nt	den y	Pitch pitch, optional value: [0- 100], with a default to 50	50
b gs	i nt	den y	Background sound of the synthetic audio 0: No background sound 1: There is a background sound	0

t te	s tri ng	den y	Text encoding format GB2312 GB K BI G 5 UNICODE (utf16 is small end) G B 18030 UT F 8	GB K
t tp	s tri ng	den y	(xtts1.0/xtts2.0 engine support only) Synthetic text type:	

parameter name	type	Must pass	description	instance
			0: Automatic judgment text class, type, if not sure will be synthesized according to the normal format text (default) 1: Specify the input text as plain text, and the system will not identify any CSSML tags 2: Specifies the text type as CSSML text 3: Specifies the text type as SSML text Default 0	
re g	s tri ng	den y	Set English pronunciation: 0: automatic judgment processing, if not sure will be processed according to the spelling of English words (default) 1: all English pronunciation 2: automatic judgment processing, if not sure will read according to the letter By default, English words	0

s ta	s tri ng	den y	Set the pause style: 0: Normal pause style (default) 1: A little frustration 2: Stunning 3: Flat and coherent 4: At one go	0
ra m	s tri ng	den y	Whether to read out punctuation: 0: do not read out all punctuation marks 1: Read out all punctuation marks	0

parameter name	type	Must pass	description	instance
rd n	s t r i n g	den y	Synthetic audio digital pronunciation 0: automatic judgment 1: full value 2: full string 3: string preferred	0
s mk	s t r i n g	den y	(Need to open permission separately) Control the format of returning pinyin annotation 0: no return 1: pure text format (gb 2312 code) 2: xml (gb2312 code) containing detailed segmentation information 3: json (gb2312 code) containing detailed segmentation information default 0	0
rh y	s t r i n g	den y	Control whether to return to the pinyin annotation. 0: Do not return to pinyin, 1: Return to pinyin (Supported engines: xtts1.0-cpu, xtts1.0-gpu, x tts 2.0-gpu, xtts 2gpu in one return) The time marked in pinyin xtts 2 Times by 5ms.0	

s fl	i nt	den y	Need to use with ue = ame, open the flow return MP3 format audio Value: 1 is turned on 0: Don't open	
s cn	i nt	den y	Scene parameters, affecting the mute length at the end of the sentence (xtts only, used by speakers)	0

parameter name	type	Must pass	description	instance
			Value: 0-no; 1-prose reading; 2-novel reading; 3-news; 4-advertising; 5-interaction	
tone_change	string	deny	Control the three or three changes Value: 0-no change; 1-change For example: "hello", which is pronounced in two cases: you [=ni 3] good [=hao 3] and you [=ni 2] good [=hao 3]	1
*em t	string	deny	netu ral:中立, naughty:调皮, conciliative:安慰, adorable:可爱, encouraged:鼓励, cheerful:高兴, apologetic:抱歉, affectionate:撒娇, beloved:宠溺, serious:严肃, puzzled:困惑, fearful:害怕, sad:悲伤, angry:生气	not have

*style	s t r i n g	den y	assist ant:助理, g entle:温柔, chat:闲 聊, w eak:虚弱, ani me:二次元, newscas t _professiona l:专业 新闻, news cast _casual:休闲 新 闻 , narrati on _profession al:专业 叙述, narra tion _relaxed:轻 松叙述 , adverti sement _upbeat	not have
--------	----------------	----------	---	----------

parameter name	type	Must pass	description	instance
			: Store advertising, adverti sement _fashio n: Fashion advertising, audiob ook _narration: Audio narration, documen tary _narratio n: Documentary film, custo mer _service: Customer service, poe try _reading: Poetry, Boy: Little boy, comm entary _ Game: Esports commentary, li vestream _ec: E- commerce live streaming, comic: Two-order yuan	
*emotion	i nt	den y	0: neutral, 1: naughty, 2: comfort, 3: lovely, 4: encourage, 5: happy, 6: sorry, 7: sajiao, 8: spoil, 9: serious, 10: confused, 11: fear, 12: sad, 13: angry	0
*emotion_ s c a l e	i nt	den y	Emotional intensity adjustment coefficient: value- 20 ~ 20, default value 0	0

- Note: The * number parameter is only supported by some x 4 speakers
- Description of the data parameter:

parameter Name	type	Must pass	description
text	string	yes	Text content, no more than 2000 characters, using base 64 encoding, not more than 13000 bytes
status	int	yes	Data status, fixed to 2

Returns the parameter example

```
{
  "code" : 0,
  "message" : "",
  "data" : {
    "audio" : "",
    "status" : 1,
    "spell" : "",
    "ced" : "370",
    "speex" : {
      "level" : "8",
      "from" : "640",
      "to" : "70"
    }
  },
  "sid" : "tts 49c 7dc 69@nc 16763de 65fc 00153e 0"
}
```

Return of the parameter description:

parameter name	type	description
code	int	Return code, 0 indicates success, other indicates exception,
message	string	descriptor
data	object	
data.audio	string	Synthetic audio clips, encoded using base 64
data.status	int	The current audio flow state, 1 indicates the synthesis, 2 indicates the end of the synthesis, the status sequence may be [1,1,..., 2] or [2] audio is very short a frame to end will not receive 1
data.spell	string	Pinyin annotation of the resultant results
data.ced	string	Synthetic audio corresponds to text progress in unit byte

parameter name	type	description
data .backup	string	Whether the composite uses the bottom speaker (only the personalized composite interface returns when the speaker returns), 0: no bottom, 1: the bottom occurs
data .speex	object	This field is returned when auue = speex-org-b, or auue = speex-org-nb The level field represents the compression level The from field represents the number of bytes before compression per frame (for reference) to represents the number of bytes after compression per frame. There may be a possible to =0 case in the tail frame.
s id	string	The id of this session is only returned at the first frame request

Synthetic multiplexing mode

The multiplexing mode is mainly designed to solve the time-consuming problem of re-establishing the connection for each server 2server call session and the overhead problem of maintaining a large number of long connections when the client concurrency is too high. The multiplexing mode supports a long connection hold and a connection concurrently initiates a multiplex session.

Multiplexing mode use:

1 Request parameter url parameter needs to be added: stream_mode=multiplex, open the multiplexing module type.in compliance with:

```
ws ://tts -api .xfyun .cn /v 2/tts ?stream _mode =multiplex
&authorization =.....
```

2 The request parameter common needs to add the parameter cid, which identifies each pass session on the current connection, and each pass session on each connection has a different cid. The server brings back the value in the response result so that the client can find the corresponding session through the field.

cid: The unique identity of each session on the connection, either a random string
A self-added string.

Example of the request parameters:

```
{  
  "common ":{  
    "ci d " : "1",  
    "app _id " : "123456"  
  },  
  "business ":{  
    "vcn " : "xiaoyan ",  
    "aue" : "raw",  
  }  
}
```

```

        "tpe " : "utf8"
    },
    "data " : {
        "status" : 2,
        "text " : "... "
    }
}

```

Example of the response results:

```

{
    "Cid": "1", // common in the value and request parameter.cid unanimous.
    "sid" : "",
    "code" : "0",
    "message " : "success ",
    "data " : {
        "audio " : "...",
        "ced" : "120",
        "status " : 1
    }
}

```

Note: 1. The client needs to ensure that the data of each frame of each session is sent to the server through the same connection, and also ensure the uniqueness of the cid on each connection. When the connection is disconnected, all unfinished sessions on the connection will be terminated. 2. Each connection is disconnected after 15s. It is recommended that clients implement the connection pool to manage the connection. 3. Suggest that the client send a close frame common at the end of the current session. The cmd = close to actively end the current session, exception
It's time to send it.close Frames are as follows:

close frame:

```

{
    "common " : {
        "app_id " : "123456",
        "cmd " : "close",
        "cid " : "1"
    },
    "data " : {

    }
}

```

Common error code

- 10163: parameter calibration failure, caused by the client parameter calibration failure, the client needs to return according

Change the request parameter with the description in the message field of

- 10313: Request parameter First frame without app_id or no app_id with api _key

mate.

- 11201: The interface usage exceeds the maximum purchase limit, please continue to use it after purchase. Internal users

Demand work orders can be submitted on itsm (itsm.iflytek .com- -> AI cloud service- -> 1001-AI capability usage application), apply for additional usage.

- 10114: Request timeout, the session time exceeds 60s, please control the session time, keep no more than

60s

frequently asked questions

- Synthetic audio sounds like text: this is usually the transmitted text coding format

Right, ensure that the encoding format of the tte parameters and the uploaded text remains consistent

- The resultant audio sounds unquickly: this is usually the wrong audio sampling rate and can be heavy

The new set adoption rate is consistent with the auf parameters before listening

Handshake & right away

The way of authentication

The client needs to summarize the requested relevant parameters using the hmac-sha 256 algorithm to generate signature and build the authorization parameters. The server resolves the authorization parameter and calculates the singature in the same way, and is equal to the signature. Equate authentication is approved.

instance url:

wss ://ws -

```
api .xfyun .cn /v 2/ivw ?authorization =aG 1hYyB 1c 2VybmFtZT 0iMTAwSU
1FIiwgYWxnb 3J pdGhtPSJobWFjLXNoYTI 1NiIsIGhlYWRLcnM 9Imhvc 3QgZGF 0ZSByZXF
1ZXN 0LWxpbnUilCBz aWduYXR 1cmU 9IlVSbnk 4M 3o 1e1JsNWF 10Dl 1YXhUL
1dGdUtWejZVNkdDdDV 25SMGdueWc 9I g %3D %3D &date =Tue %2C +18+Dec
+2018+09%3A 08%3A 49+UTC &host =10.1.87.70%3A 8000
```

Authentication parameters

parameter	type	must	explain	example
host	string	yes	Request host	iat -api .xfyun .cn
date	string	yes	Current Timestamp , RFC 1123 format (Mon, 02 Jan 2006 15:04:05 GMT)	Fri , 18 Jan 2019 07:21:29 UTC
authorization	string	yes	Signature-related information encoded using the base 64 (Signature calculation based on hmac-sha 256)	aG 1hYyB 1c 2VybmFtZT 0iNDNiMTh iMWU 4MGNmMzlmODFiMGEyMjk xNjE 5YTU 1OWUiLCBhbGdvcml 0a G 09lmhtYWMtc 2hhMjU 2liwgaGVh ZGVycz 0iaG 9zdCBkYXRlIHJlcXVlc 3QtbGluZSIsIHNPZ 25hdHVyZT 0iY W 4wQ 2xYdGRQTG 92ZDdFT 1phL 1p 2VzNXT 0ZRUIBiU 0NqMTViRzZ NcXFSZz 0i

Signature generation formula

```
api_key ="${api_key }",algorithm ="hmac -sha 256",headers ="host  
date request -line ",signature ="${signature }"
```

Explanation: api _ key: open platform application apikey
algorithm: signature calculation algorithm, support hmac-sha 256
headers: need to participate in the signature parameters, need at least host date request-line this three signature: calculated signature

Detailed authentication process

1. Build the string to be signed signature _ origin _ str

The signature _ origin _ str format is:

```
host : ${host }\ndate : ${date }\n ${method } ${path } HTTP /1.1
```

- Host: The requested host header for example. iat -api .xfyun .cn
- Date: the current timestamp, the format is Wed, 08 Jun 2022 08:12:15 UTC

•method: The request method supports the GET POST DELETE PATCH PUT required and current requests for http

method Consistent

•ath: path of request. Not need to include the query string part ('?'And the following part), such as the url of

'/v 2/iat ? For a =b & c =d ', we should only take the ' / v 2 / iat'part as the value of the path.

Note: ':' There is a space behind it, '\ n' is the envelope, and HTTP / 1.1 is the http protocol version number. If the client uses the HTTP / 1.0 protocol, the value should change to HTTP / 1.0

2. Calculate the signat ure

The calculation method is to calculate the signature of the signature _ origin _ str using the hmac-sha 256 algorithm and do base 64 coding.

```
signature = base 64(hmac -  
sha 256(${signature _origin _str },${api _secret })))
```

Where api _ secret is the key part of the key pair acquired by the platform side, and signature _ origin _ str is the parameter of the previous splicing step

3. Splicing authorization _ row parameter

```
authorization _raw = api _key ="${api _key }",algorithm ="hmac -  
sha 256",headers ="host date request -line ",signature ="${signature  
}"
```

4. Build the authoirzation parameter

```
authorization = base 64(authorization _raw )
```

5. Splice request url

```
url = ${api _url }?  
authorization =${authorization }&host =${host }&date =${date }
```

Signature generation example:

With the following request, the request address is ws: // iat-api.xfyun . The cn / v 2 / iat, the request method is GET, User's api _ key = 5 ccdf 2b 4d 1b 5cdf 81846697bf 8bcd 05d, api _secret = B 00TFRS 9KDCfTrdX 5JQwhVSXaFoHly 34, Current time is: Web, 08 Jun 2022 09:00:06 UTC 1 then signature _ origin _ str should be assembled like this

```
host : iat -api .xfyun .cn  
date : Wed , 08 Jun 2022 09:00:06 UTC  
GET /v 2/iat HTTP /1.1
```

2 using signature _ origin _ str, and the calculated signature =

VhEap 7PkvX 7ujjx 8DjBtkRZFwQDIEOc 62EM +M 9N +pf 8=

3 then authorization _raw is

```
api_key ="5ccdf 2b 4d 1b 5cdf 81846697bf 8bcd 05d ",algorithm
="hmac - sha 256",headers ="host date request -
line ",signature ="VhEap 7PkvX 7ujjx 8DjBtkRZFwQDIEOc 62EM +M 9N +pf 8="
```

4 authorization parameter is the results after base 64 encoding of authorization _raw

```
YXBpX 2tleT 0iNWNjZGYyYjRkMWI 1Y 2RmODE 4NDY 20TdiZjhiY 2QwNWQilGFsZ
29ya XRobT 0iaG 1hYy 1zaGEyNTYilGhlyWRlcnM 9Imhvc 3QgZGF 0ZSByZXF
1ZXN 0LWxpbm UiLHNpZ 25hdHVyZT 0iSk 9CYTBTv 3d 0WFVYTjU 4ZWlsSENKa
21oZnVQVHpvWXcrSHp 6UXpKa 3ExST 0i
```

5 The final request url is for

```
ws ://iat -api .xfyun .cn /v 2/iat ?authorization =YXBpX 2tleT
0iNWNjZGYyYjRkMWI 1Y 2RmODE 4NDY 20TdiZjhiY 2Q wNWQilGFsZ 29yaXRobT
0iaG 1hYy 1zaGEyNTYilGhlyWRlcnM 9Imhvc 3QgZGF 0ZSBy ZXF 1ZXN
0LWxpbmUiLHNpZ 25hdHVyZT 0iVmhfYXA 3UGt 2WDd 1amp 40ERqQnRrUlpGd
1FESUVPYzYyRU 0rTTlOK 3BmOD 0i &date =Wed %2C +08+Jun +2022+09%3A
00%3A 06+ UTC &host =iat -api .xfyun .cn
```

Users can use this example to verify that their signature algorithm implements correctly

Key function description

function name	explain
hmac-256	A standard signature algorithm, providing the key and the data to be signed, can calculate the signature summary, the result of sha 256 calculation is the original byte, cannot be encoded.
base 64	A standard encoding method for encoding bytes into visible strings, noting the use of a standard base 64 instead of a base64 _url

Handshake failure return example:

```
HTTP /1.1 403 Forbidden
Date : Thu , 06 Dec 2018 07:55:16 GMT
Content -Length : 116
Content -Type : text /plain ; charset =utf -8
```

```
{
"message ": "HMAC signature does not match "
}
```

List of language speakers

The following are some languages and speakers, for your reference only

classification of language	vc n	Whether to open
the Chinese language	x 4_lingxiaoyue _oral	yes
English	x 4_EnUs _Lara _profnews	yes
Malay	x 2_malayrole	yes
sun	qianhui (Female) The x 2_zhongcun (female)	deny
Indonesia	X. x 2 _ idid _ kris (female)	deny
very soon	al labent The x 2 _ rrulu _ keshu (female)	deny
Italian	_ x 2 _ ltlit _ Anna (female) _ x 2 _ ltlit _ Luca (male)	deny
、 、 、	、 、 、	、 、 、